Equipo 7:

Luis Enrique Almaguer Castillo - 2143523

Héctor Alexander Tovar González - 2144664

Lizzie Anette Mireles Tejada - 2144591

Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad: U-6	**Actividad**: PIA
Fecha: 06/11/2026	
Repositorio de GitHub: https://github.com/Lamaguer/NovaRentU6 **Sistema desplegado:** https://novarent.onrender.com/ **Swagger/OpenAPI:** https://novarent.onrender.com/docs	

Objetivo:

Organizar y mostrar documentos de Arquitectura y fundamentación del producto final de la empresa ficticia NovaRent dentro de un reporte final, este contendrá; BRD, Arquitectura, evidencias y conclusiones.

INDICE

BRD 3
1. Contexto 3
2. Problemas 3
3. Actores 4
4. Alcance 4
5.Requerimientos y Necesidades de Negocio 5
6. Restricciones y Reglas de Negocio 5
7. Supuestos 6
Arquitectura - ADR's 7
Arquitectura por capas 11
Modelo de Dominio (DDD) 12
Máquina de estado - Vehículo 13
Máquina de estado - RESERVACIÓN 14
Máquina de estado - RENTA 15
Especificaciones (RN-01, RN-02, RN-03) 16
Secuencia: Flujo principal y excepcional 16
Secuencia: Reservar y bloquear inventario 17
BPMN 18
Patrones de diseño seleccionados 19
Principios SOLID aplicados 20
Evidencias 22
Docker Test - 1 22
Docker test - 2 24
Docker Test - 3 25
Actions en verde después de pushes 26
Render afirmando que la página está levantada/live 26
Nova Rent SWAGGER - LIVE 27
NovaRent principal - LIVE 27
Renta Reservada 28
Registración de Venta 29
Estado después de renta 29
Renta Extendida 30
Docker corriendo 30
Limitaciones conocidas 31
Bitácora de IA 32
Conclusiones 32

BRD

Campo	Valor
Documento	BRD — Business Requirements Document
Proyecto	Sistema de Gestión Operativa NovaRent
Versión	1.1
Estado	Baseline de requerimientos de negocio
Fecha	04 de Noviembre de 2026
Propietario	NovaRent

1. Contexto

NovaRent renta automóviles por día. El cliente reserva una categoría; la unidad física se asigna hasta la entrega en sucursal. Genera ingresos extra con seguros, conductores adicionales y otros servicios.

2. Problemas

Daños mal documentados · cobros confusos · inventario desactualizado (sobreventas) · retraso en avisar a limpieza.

3. Actores

Rol (Actor)	Responsabilidad Operativa
Cliente	Inicia el proceso al hacer la reservación, entrega sus documentos y recoge/devuelve el auto.
Agente de Renta	Atiende en mostrador, revisa los documentos del cliente, realiza los cobros y registra la entrega y devolución del auto.
Personal de Patio	Mueve los autos en la sucursal y ayuda con la revisión visual rápida al entregar y recibir los vehículos.
Personal de Limpieza	Lava y prepara el auto devuelto para que pueda rentarse nuevamente.
Administración	Supervisa que todo funcione bien, resuelve quejas difíciles de los clientes y toma decisiones sobre los autos.

4. Alcance

- Reservaciones y disponibilidad real por categoría.
- Registro digital de entrega (documentos y checklist).
- Cálculo automático de cargos extra.
- Seguimiento del estado del vehículo (Rentado, En Limpieza, Disponible)
- KPIs. Fuera de alcance: cancelación de reservaciones, seguros y conductores adicionales, pagos reales (se usa un adaptador simulado).

5.Requerimientos y Necesidades de Negocio

Nivel de Prioridad	Nivel Crítico	Descripción
1	Alto	**BR-01 Checklist Digital**: El sistema debe tener una aplicación (en tablet o móvil) para registrar los daños del auto con fotos al entregarlo y recibirlo.
2	Alto	**BR-02 Disponibilidad Real:** El sistema de reservaciones debe conectarse al inventario real para no permitir rentar autos si la categoría ya está llena.
3	Alto	**BR-03 Motor de Cargos:** El sistema debe calcular automáticamente cuánto cobrar de penalización basándose en una lista de precios oficial de la empresa.
4	Bajo	**BR-04 Avisos a Limpieza:** El sistema debe mandar una alerta o mostrar en pantalla la limpieza cuando un auto es devuelto para que lo preparen rápido.

6. Restricciones y Reglas de Negocio

Restricción	Descripción
RN-01 Categoría Superior	Si no hay autos de la categoría reservada, se entrega una categoría superior sin costo extra (se conserva la tarifa original).
RN-02 Requisitos de Entrega	No se registra la entrega sin identificación, licencia vigente y pago/tarjeta. El checklist inicial también es requisito previo (glosario D023, US-07).
RN-03 Extensión de Días	Una extensión solo se permite si la categoría tiene lugar en fechas nuevas. Se valida contra el inventario de la categoría.

7. Supuestos

- Las sucursales tienen buena conexión a internet para usar el sistema sin problemas.
- La empresa ya tiene una lista de precios fijos y aprobados para cobrar daños y litros de gasolina.
- Tarifario ficticio de demostración.

8. Criterios de Éxito (KPIs)

Sabremos que el proyecto fue un éxito si en los primeros 3 meses logramos lo siguiente:

- Tiempos más rápidos: Reducir un 40% el tiempo desde que el cliente devuelve el auto hasta que la limpieza lo tiene listo para otra renta.
- Menos quejas: Reducir un 60% las quejas de clientes por cobros de daños que no reconocen.
- Cero sobreventas: Bajar a 0% los casos donde el cliente llega a la sucursal y no tenemos un auto disponible para él

Arquitectura - ADR's

Titulo	ADR-001 Arquitectura en capas con DDD
Contexto	El PIA exige DDD, Specification, State Machine y API REST, con src/ dividido en domain, application, infrastructure y api.
Alternativas	Portar el esqueleto Java del Equipo 7 (descartado: no hay tiempo y las pruebas/CI del curso están en Python).
Decisión	Python 3.11 + FastAPI + SQLAlchemy + PostgreSQL. El dominio no depende de frameworks; la API traduce errores de dominio a mensajes en español.
Consecuencias	Dominio probado de forma aislada; el esqueleto Java queda como referencia de modelado.

Titulo	ADR-002 El checklist inicial es requisito previo de la entrega
Contexto	El BPMN genera el checklist después de entregar; el glosario (D023) y US-07 lo listan como requisito para registrar la entrega.
Alternativas	Generarlo después (contradice US-07 y la regla RN-02).
Decisión	Se exige antes de entregar (ChecklistSpec dentro de requisitos_completos)
Consecuencias	El BPMN debe actualizarse en el reporte final.

Titulo	ADR-003 Inventario por categoría, medido día por día; extensión validada contra la categoría
Contexto	Se reserva una categoría, no una unidad (D002, D006); RN-03 habla de "ese auto".
Alternativas	Contar traslapes entre reservas (sobrerrechaza); validar contra una unidad física (no existe al reservar).
Decisión	InventarioCategoria cuenta lugares apartados por cada día del periodo [inicio, fin). Una reserva que empieza el día en que otra termina no choca. La extensión excluye la propia reserva al contar.
Consecuencias	Se evitan sobreventas sin bloquear casos válidos.

Titulo	ADR-004 Tarifario ficticio y supuestos de cálculo
Contexto	El BRD supone una lista de precios pero no incluye valores.
Alternativas	Esperar la lista oficial del cliente (no disponible para la entrega).
Decisión	Valores de demostración en src/domain/politicas.py: tarifa diaria 500/650/900/1400; retraso $150/h (hora límite 12:00); 200 km/día incluidos y $4/km extra; combustible $30/L con tanque de 45 L en octavos; $500 por daño nuevo. Todo en Decimal.
Consecuencias	Reemplazables sin tocar la lógica. Deben sustituirse por los valores oficiales del cliente.

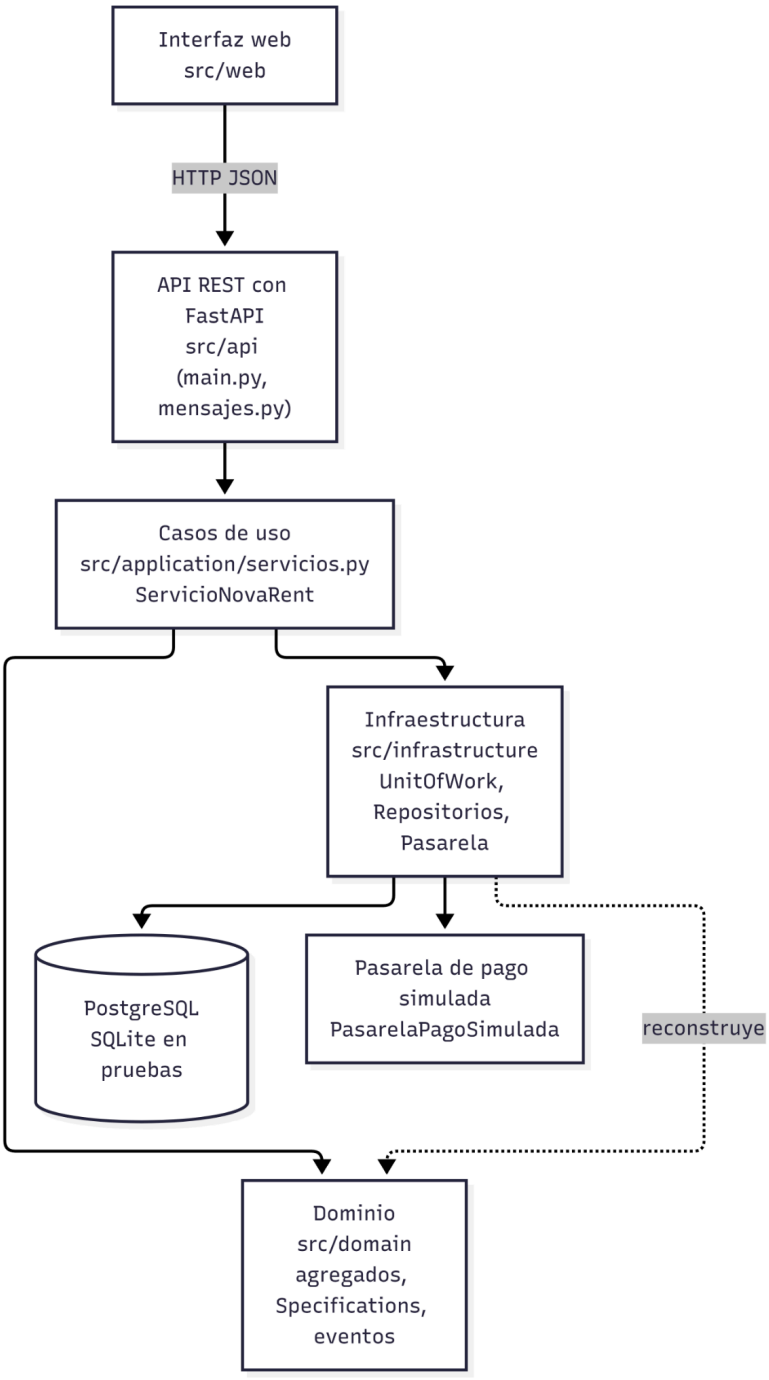
Titulo	ADR-005 Despliegue con Docker, Docker Compose y Render con PostgreSQL
Contexto	El PIA exige un entorno reproducible (DB + App + Tests con Docker Compose), configuración separada del código (12-Factor) y un despliegue funcional con Swagger activo en producción. El equipo necesita una opción gratuita que cualquier integrante pueda operar.
Alternativas	• Supabase para la base de datos: descartada por ahora, porque exige conexión por pooler (IPv6) y activar RLS; Render permite declarar app y base en un mismo archivo. • Railway o Fly.io: sin ventaja clara para este alcance y con menos práctica del equipo. • Interfaz y API por separado: duplica despliegue y configuración sin beneficio para la demostración.
Decisión	• Un solo contenedor sirve la API (FastAPI) y la interfaz web estática (src/web), para no mantener dos despliegues. • El Dockerfile tiene etapas: base, test (herramientas de prueba) y prod, que va al final para que sea la que se construye por defecto. • docker-compose.yml levanta db y web; con el perfil tests agrega db-test y tests. Las pruebas usan una base separada, porque borran y recrean tablas. • La configuración llega por variables de entorno (DATABASE_URL, APP_ENV, PORT); .env.example no contiene secretos reales y .env está ignorado por Git. • En producción se despliega en Render con render.yaml (Blueprint): servicio web con Docker y base PostgreSQL, ambos en plan gratuito, con autoDeploy en cada push a main y healthCheckPath: /health.

Consecuencias	(+) Cualquier integrante levanta el sistema con docker compose up, y el despliegue se actualiza solo al hacer push. (−) El servicio gratuito se duerme tras minutos sin tráfico y la primera visita tarda cerca de un minuto; conviene abrir la URL antes de presentar. (−) La base gratuita de Render caduca a los 30 días de crearse; la fecha de creación debe anotarse en el reporte. (−) No hay migraciones: las tablas se crean al arrancar, así que un cambio de columna exige recrear la base.

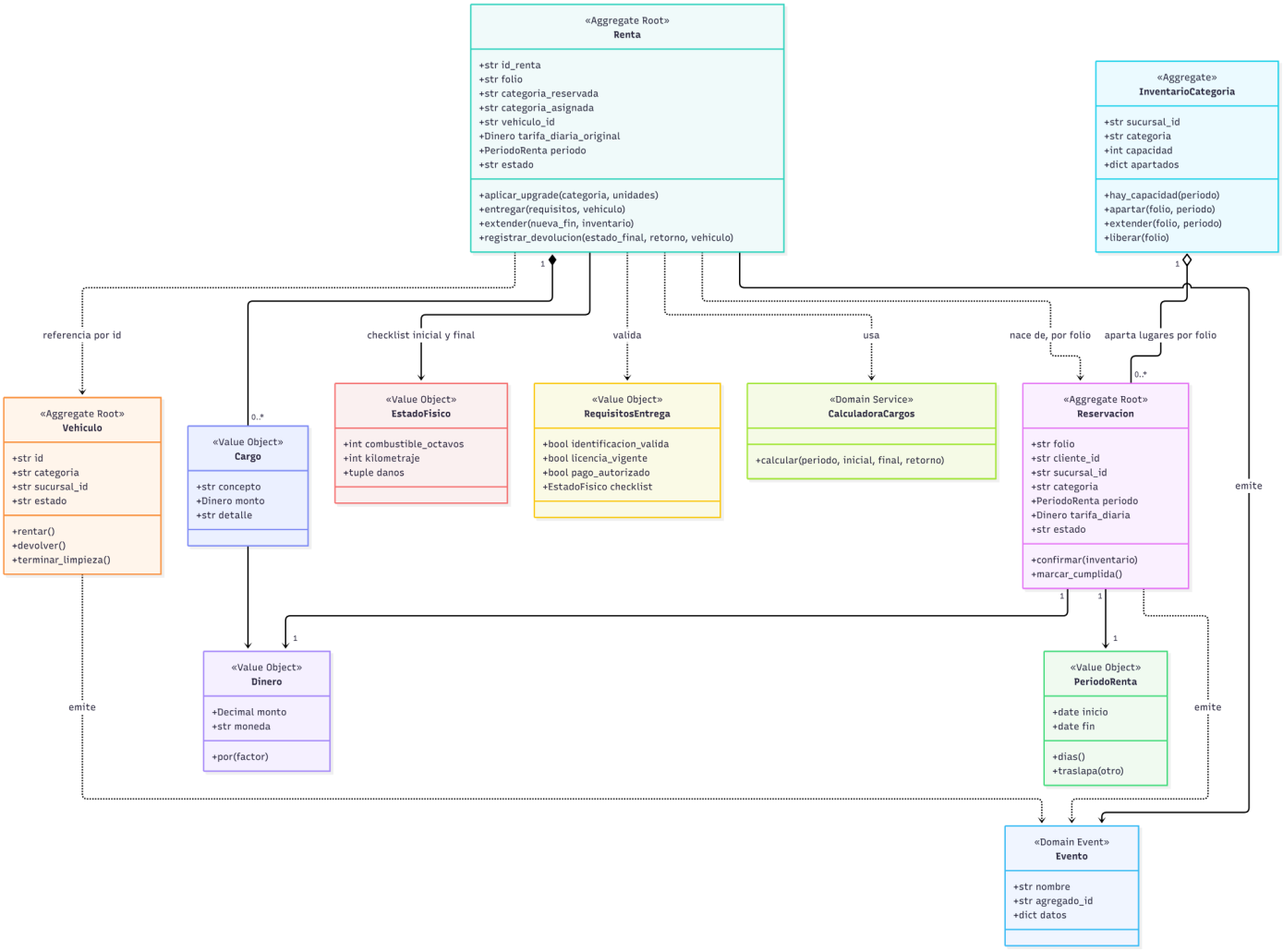
Titulo	ADR-006 Persistencia con Unit of Work, repositorios por agregado y bloqueo de fila
Contexto	Cada caso de uso toca varios agregados (por ejemplo, entregar actualiza la renta, el vehículo y los eventos). Si una regla de negocio falla a la mitad, no puede quedar nada guardado. Además, BR-02 exige cero sobreventas aunque dos reservaciones lleguen al mismo tiempo.
Alternativas	• Un único servicio que haga commit en cada método: no garantiza atomicidad entre agregados. • Guardar el dinero como texto o float: pierde precisión o impide sumar en la base. • Bloqueo optimista con versión de fila: más código para un riesgo que el bloqueo de fila ya cubre.
Decisión	• UnitOfWork abre una sesión por caso de uso: hace commit solo si todo sale bien y rollback si el dominio lanza un error. • Hay un repositorio por agregado (reservaciones, rentas, vehículos, inventarios, eventos). Reconstruyen los objetos de dominio desde la base, así el dominio no conoce SQLAlchemy. • El inventario de cada categoría se carga con SELECT ... FOR UPDATE (bloquear=True) al reservar, para que dos reservaciones simultáneas sobre la misma categoría se atiendan una tras otra. • El dinero se guarda como Numeric(12,2) y se maneja como Decimal, nunca como float. • Los eventos de dominio se persisten en una tabla y alimentan los indicadores (US-15). • Las pruebas de integración corren contra SQLite en memoria local y contra PostgreSQL en el pipeline de CI.
Consecuencias	(+) Un rechazo del dominio no deja datos a medias (prueba: test_br02_sobreventa_rechazada_y_nada_se_guarda). (+) La sobreventa concurrente se evita en PostgreSQL. (−) En SQLite el bloqueo de fila no tiene efecto; la garantía de

	concurrencia solo se valida en PostgreSQL. (−) Cada caso de uso abre su propia transacción, así que no hay transacciones largas entre varias pantallas.

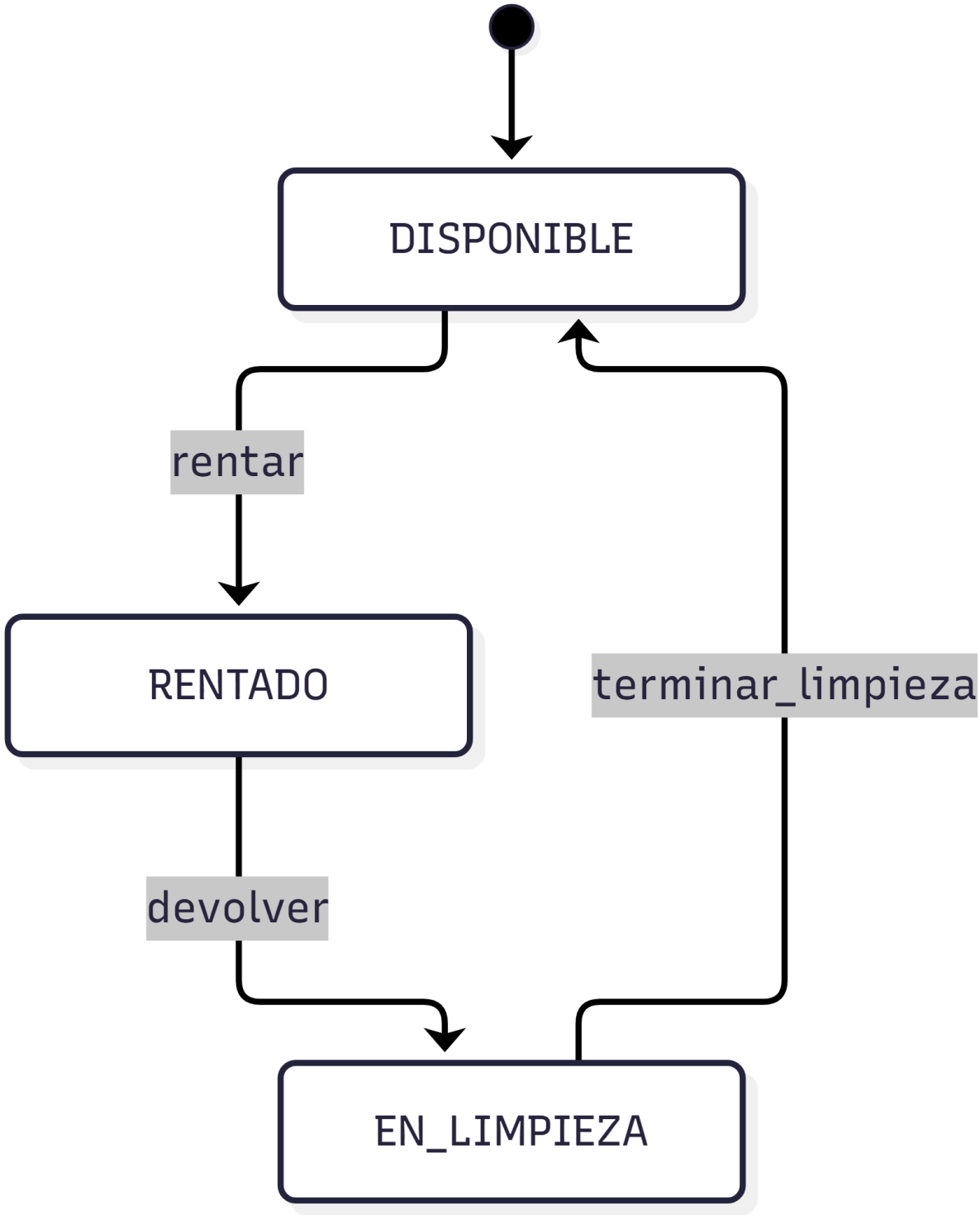
Arquitectura por capas



Modelo de Dominio (DDD)



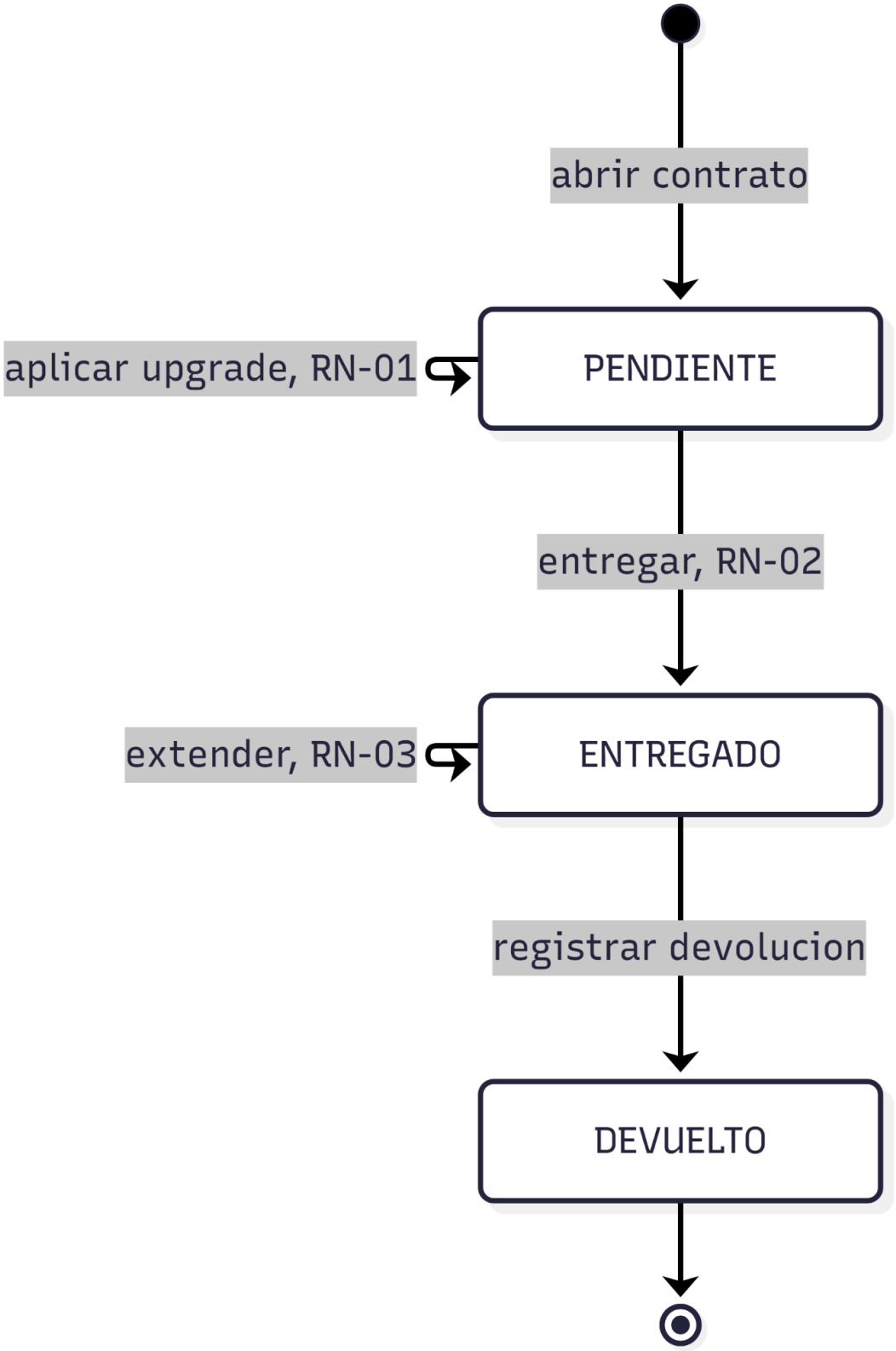
Máquina de estado - Vehículo



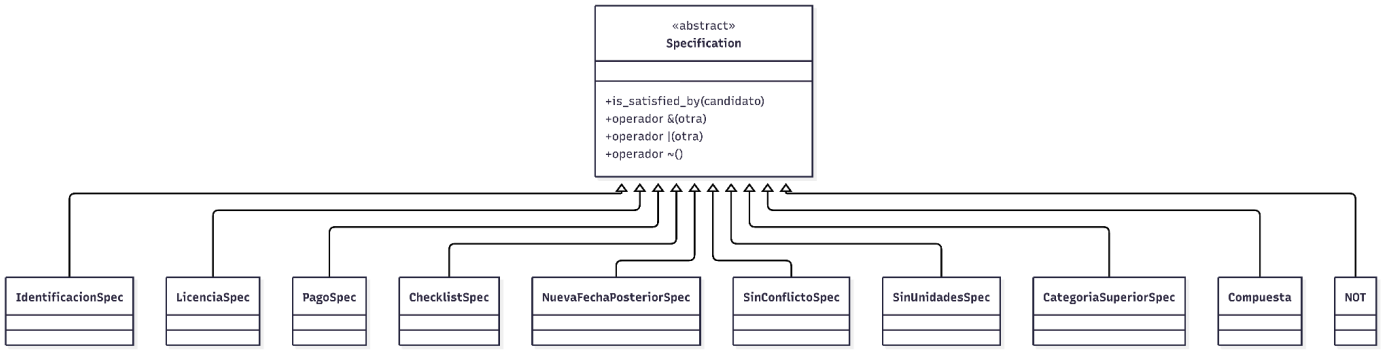
Máquina de estado - RESERVACIÓN



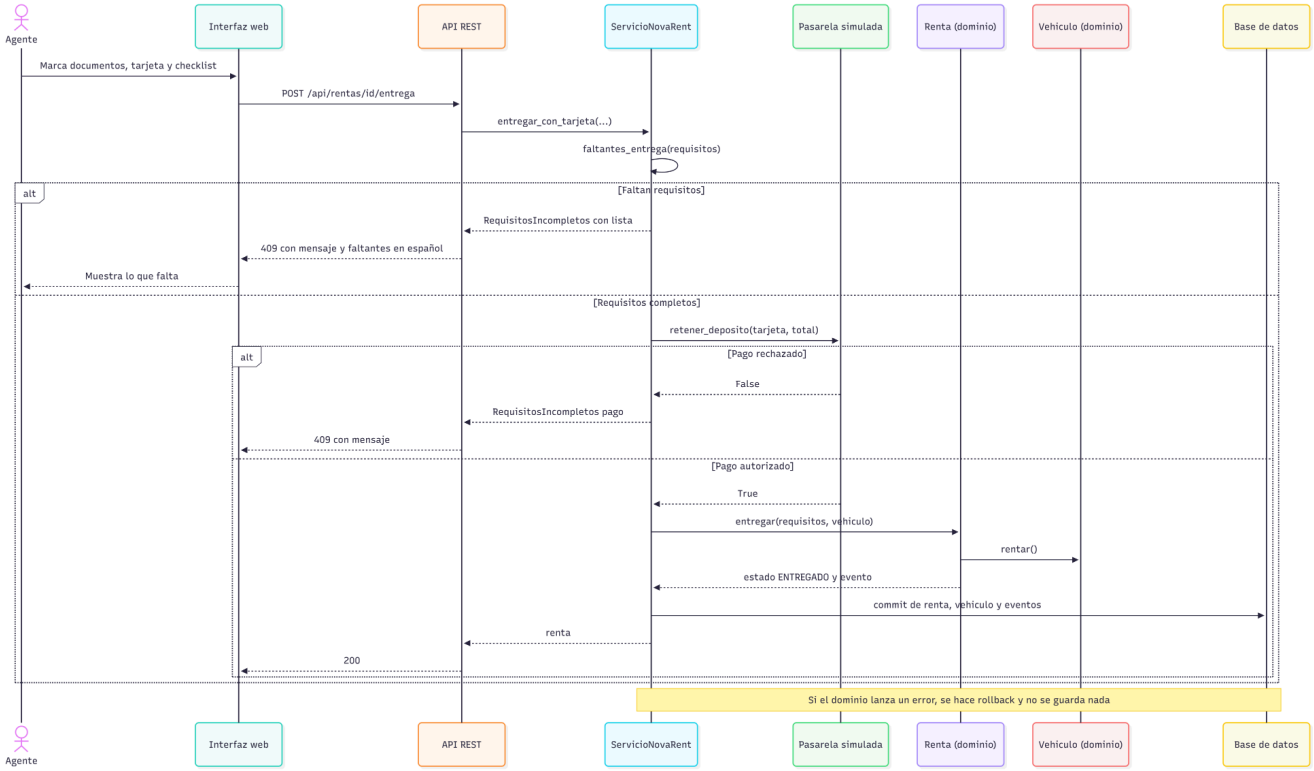
Máquina de estado - RENTA



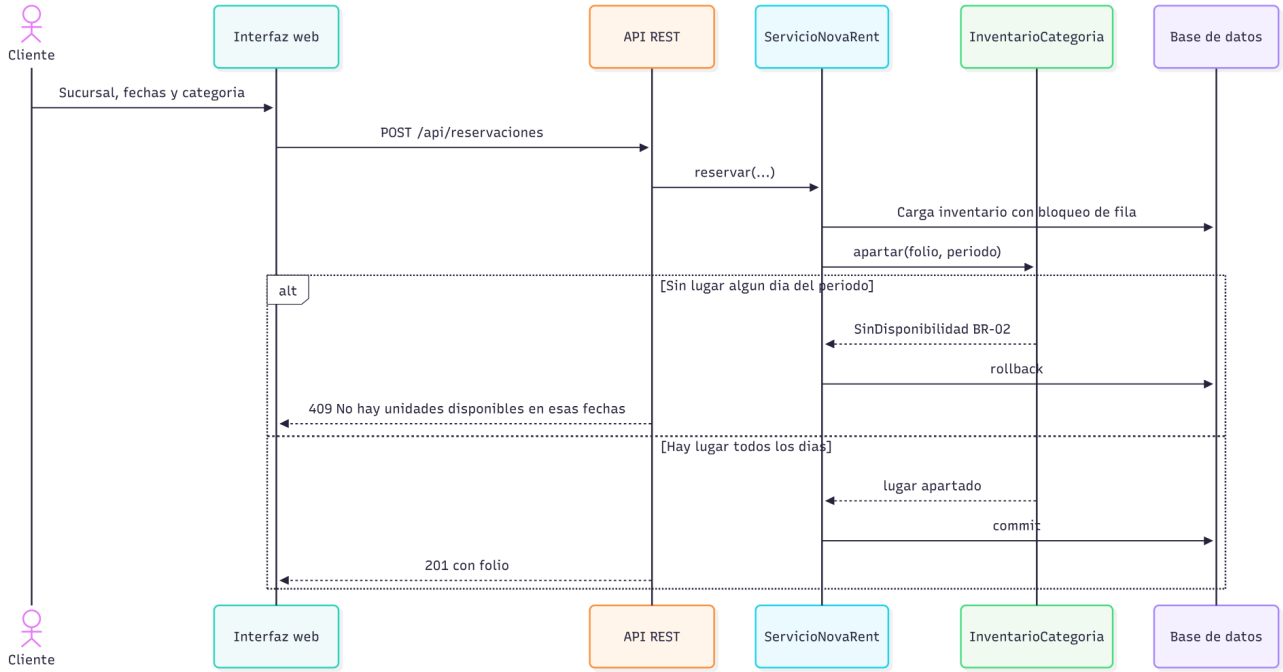
Especificaciones (RN-01, RN-02, RN-03)



Secuencia: Flujo principal y excepcional

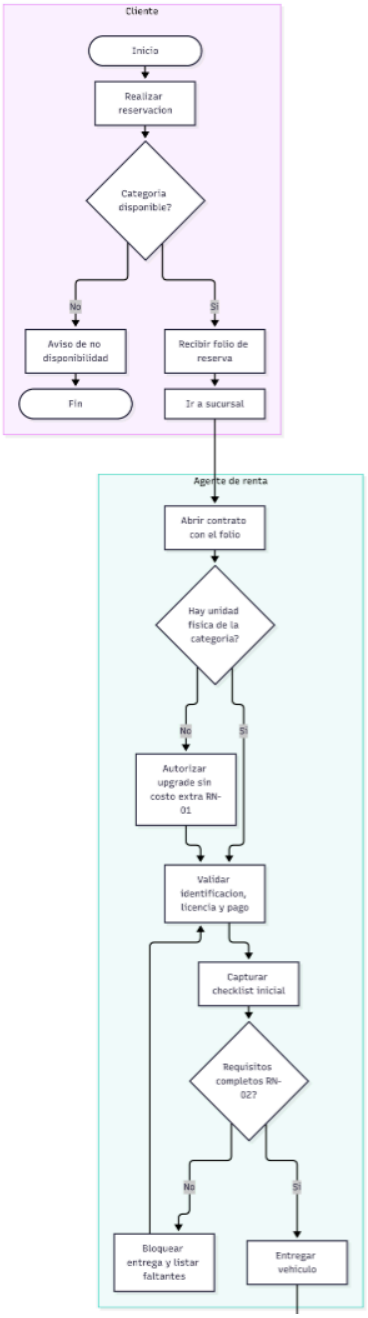


Secuencia: Reservar y bloquear inventario

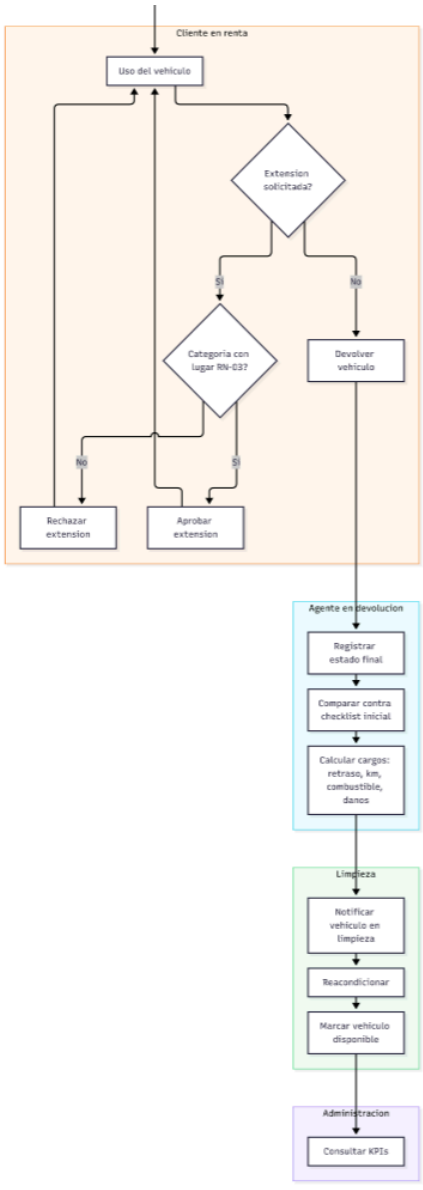


BPMN

PT. 1


Cliente
Inicio
Realizar reservacion
Categoria disponible?
No
Si
Aviso de no disponibilidad
Recibir folio de reserva
Fin
Ir a sucursal
Agente de renta
Abrir contrato con el folio
Hay unidad fisica de la categoria?
No
Si
Autorizar upgrade sin costo extra RN-01
Validar identificacion, licencia y pago
Capturar checklist inicial
Requisitos completos RN-02?
No
Si
Bloquear entrega y listar faltantes
Entregar vehiculo

PT.2


Cliente en renta
Uso del vehiculo
Extension solicitada?
Si
No
Categoria con lugar RN-03?
Devolver vehiculo
No
Si
Rechazar extension
Aprobar extension
Agente en devolucion
Registrar estado final
Comparar contra checklist inicial
Calcular cargos: retraso, km, combustible, danos
Limpieza
Notificar vehiculo en limpieza
Reacondicionar
Marcar vehiculo disponible
Administracion
Consultar KPIs

Patrones de diseño seleccionados

Patrón	Dónde	Por qué
Aggregate / Entity (DDD)	Reservacion, Renta, Vehiculo, InventarioCategoria	Cada agregado protege sus invariantes y se guarda en una transacción.
Value Object	Dinero, PeriodoRenta, EstadoFisico, RequisitosEntrega, Cargo	Inmutables y validados al crearse: no puede existir un monto negativo ni un periodo invertido.
Specification	src/domain/specifications.py	RN-01, RN-02 y RN-03 son condiciones que se combinan; así se prueban por separado y la entrega puede listar exactamente qué falta.
State Machine	Vehiculo (tabla de transiciones), Renta, Reservacion	El BRD define estados del vehículo; las transiciones inválidas lanzan error.
Domain Events	Evento y 8 eventos persistidos	Alimentan la limpieza (BR-04) y los KPIs (US-15) sin acoplar módulos.
Unit of Work	UnitOfWork	Una transacción por caso de uso: commit si todo sale bien, rollback si el dominio rechaza.
Repository	Repositorio*	Reconstruye agregados desde la base y los guarda; el dominio no conoce SQLAlchemy.

Adapter	PasarelaPagoSimulada	El BRD no incluye un proveedor de pagos real; el adaptador permite simular cobros sin mover dinero.
Domain Service	CalculadoraCargos	El cálculo involucra varias entidades y no pertenece a ninguna.

Principios SOLID aplicados

Principio	Evidencia en el código	Matiz
S Responsabilidad única	CalculadoraCargos solo calcula cargos; InventarioCategoria solo controla capacidad; mensajes.py solo traduce errores; el dominio no sabe de HTTP ni de SQL.	ServicioNovaRent concentra todos los casos de uso en una clase: podría dividirse por módulo (reservas, rentas, limpieza).
O Abierto/cerrado	Una regla nueva se agrega como otra Specification y se combina con &, | o ~ sin tocar las existentes. El tarifario (politicas.py) cambia sin modificar la lógica de cargos.	—
L Sustitución de Liskov	Todas las Specifications (LicenciaSpec, Compuesta, Not...) se usan donde se espera una Specification.	—
I Segregación de interfaces	La pasarela expone un solo método (retener_deposito); Specification expone solo is_satisfied_by.	—
D Inversión de	El dominio no importa	La pasarela tiene un valor

dependencias	infraestructura; ServicioNovaRent recibe la session_factory y la pasarela por constructor, y en pruebas se sustituyen.	por defecto concreto (PasarelaPagoSimulada) y no hay una interfaz explícita; un Protocol lo formalizaría.

Evidencias

Docker Test - 1

Windows PowerShell

```
 => [tests internal] load .dockerignore                                                    0.1s
 => => transferring context: 204B                                                          0.0s
 => [tests base 1/2] FROM docker.io/library/python:3.11-slim@sha256:db20d8506e2f15b6db39b40b6015ca46bae855dec  33.6s
 => => resolve docker.io/library/python:3.11-slim@sha256:db20d8506e2f15b6db39b40b6015ca46bae855dec6dd326d462cf  0.2s
[+] Building 282.4s (21/21) FINISHED
 => [internal] load local bake definitions                                                 0.1s
 => => reading from stdin 983B                                                             0.1s
 => [web internal] load build definition from Dockerfile                                   0.2s
 => => transferring dockerfile: 1.14kB                                                     0.1s
 => [tests internal] load metadata for docker.io/library/python:3.11-slim                  3.3s
 => [auth] library/python:pull token for registry-1.docker.io                              0.0s
 => [tests internal] load .dockerignore                                                    0.1s
 => => transferring context: 204B                                                          0.0s
 => [tests base 1/2] FROM docker.io/library/python:3.11-slim@sha256:db20d8506e2f15b6db39b40b6015ca46bae855dec  33.6s
 => => resolve docker.io/library/python:3.11-slim@sha256:db20d8506e2f15b6db39b40b6015ca46bae855dec6dd326d462cf  0.2s
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5  22.3s
 => => extracting sha256:ee5aaeba9945927662bea2702989dfd74d9273796415b896363e6a9c44d9a8ce  1.2s
 => => extracting sha256:24ff04c921070df4f4dc987278926d5dabde59d57b55ae3665439331f79af932  9.2s
 => => extracting sha256:361db81715472ecba5e656c314c299bd9453090204831595 96a8d123c2b1d580  0.1s
 => [web internal] load build context                                                      0.5s
 => => transferring context: 112.86kB                                                      0.2s
 => [tests internal] load build context                                                    0.2s
 => => transferring context: 92.06kB                                                       0.2s
 => [tests base 2/2] WORKDIR /app                                                          0.2s
 => [web prod 1/4] COPY requirements-prod.txt .                                            0.2s
 => [tests test 1/4] COPY requirements.txt pyproject.toml ./                               0.2s
 => [web prod 2/4] RUN pip install --no-cache-dir -r requirements-prod.txt                75.6s
 => [tests test 2/4] RUN pip install --no-cache-dir -r requirements.txt                  153.5s
 => [web prod 3/4] COPY src ./src                                                          0.8s
 => [web prod 4/4] RUN useradd --create-home --uid 10001 novarent                          4.0s
 => [web] exporting to image                                                              55.6s
 => => exporting layers                                                                   18.0s
 => => exporting manifest sha256:692d2cf713ef23fa4cb8b688570c59f963e6ce855f4db897ceffd0d196cbdc3b  0.2s
 => => exporting config sha256:c9fb989c8b220af8ac0767956cf82b91a2591464511a697164b108a514f3bb1f  0.1s
 => => exporting attestation manifest sha256:dd9c9fb0fc592a7bb0e6578c7f96160a3ed767f4b8ff01993469041ead9e5eb5  0.1s
 => => exporting manifest list sha256:035382a9d67fddd80ff3931b744b54c56a84e0c0bf1ae5b8f15a119db85978ed  0.1s
 => => naming to docker.io/library/novarentu6-web:latest                                   0.6s
 => => unpacking to docker.io/library/novarentu6-web:latest                               33.6s
 => [web] resolving provenance for metadata file                                           0.9s
 => [tests test 3/4] COPY src ./src                                                        1.8s
 => [tests test 4/4] COPY tests ./tests                                                    3.3s
 => [tests] exporting to image                                                            78.1s
 => => exporting layers                                                                   30.4s
 => => exporting manifest sha256:cf00b6eecc9b8277095a570ef2199ed447d8233cf2d03ed73aef650b7d359f2d  0.2s
 => => exporting config sha256:889c29f77be6d6d86a3054582a920ac7eaa21b3d1a159e7c98b5bda6c820945e  0.2s
 => => exporting attestation manifest sha256:7c81f91eba3f073e66d522431a9aba3e69266706817f91309ba955d0c87df74a  1.3s
 => => exporting manifest list sha256:96b878e87b5a63918ed67118d7ec03d94af2b0220b559b8411733db75fc01a11  0.2s
 => => naming to docker.io/library/novarentu6-tests:latest                                 0.1s
 => => unpacking to docker.io/library/novarentu6-tests:latest                             44.6s
 => [tests] resolving provenance for metadata file                                         0.5s
[+] build 2/2
 ✔ Image novarentu6-tests Built                                                          283.6s
 ✔ Image novarentu6-web   Built                                                          283.4s
PS C:\Users\Acer\Desktop\NovaRentU6>
```

Docker test - 2

```
tests/integration/test_api.py::test_flujo_feliz_por_api PASSED                                                [  1%]
tests/integration/test_api.py::test_us01_disponibilidad_oculta_categorias_llenas PASSED                       [  3%]
tests/integration/test_api.py::test_errores_llegan_en_lenguaje_de_cliente PASSED                              [  5%]
tests/integration/test_api.py::test_entrega_incompleta_devuelve_faltantes_legibles PASSED                     [  6%]
tests/integration/test_api.py::test_us05_pago_rechazado_por_pasarela_simulada PASSED                          [  8%]
tests/integration/test_api.py::test_upgrade_y_extension_por_api PASSED                                        [ 10%]
tests/integration/test_api.py::test_extension_bloqueada_se_explica_al_cliente PASSED                          [ 11%]
tests/integration/test_api.py::test_no_encontrado_validacion_y_estado_sin_tecnicismos PASSED                  [ 13%]
tests/integration/test_api.py::test_la_interfaz_web_se_sirve_desde_la_raiz PASSED                             [ 15%]
tests/integration/test_servicios.py::test_us02_reservacion_confirmada_sobrevive_a_una_nueva_transaccion PASSED [ 16%]
tests/integration/test_servicios.py::test_br02_sobreventa_rechazada_y_nada_se_guarda PASSED                   [ 18%]
tests/integration/test_servicios.py::test_br02_limite_dia_de_devolucion_se_puede_reservar_de_nuevo PASSED     [ 20%]
tests/integration/test_servicios.py::test_folio_duplicado_y_categoria_invalida PASSED                         [ 21%]
tests/integration/test_servicios.py::test_reservacion_sin_inventario_no_existe PASSED                         [ 23%]
tests/integration/test_servicios.py::test_confirmar_despues_de_crear_pendiente PASSED                         [ 25%]
tests/integration/test_servicios.py::test_ciclo_completo_con_cargos_persistidos PASSED                        [ 26%]
tests/integration/test_servicios.py::test_eventos_de_dominio_quedan_en_la_bitacora PASSED                     [ 28%]
tests/integration/test_servicios.py::test_rn02_entrega_bloqueada_no_cambia_nada PASSED                        [ 30%]
tests/integration/test_servicios.py::test_entrega_sin_vehiculos_disponibles PASSED                            [ 31%]
tests/integration/test_servicios.py::test_devolver_renta_no_entregada_y_entregar_dos_veces PASSED             [ 33%]
tests/integration/test_servicios.py::test_iniciar_renta_de_reservacion_pendiente_o_ya_cumplida PASSED         [ 35%]
tests/integration/test_servicios.py::test_terminar_limpieza_de_vehiculo_disponible_y_ids_inexistentes PASSED  [ 36%]
tests/integration/test_servicios.py::test_rn03_extension_con_y_sin_disponibilidad PASSED                      [ 38%]
tests/integration/test_servicios.py::test_rn01_upgrade_solo_sin_unidades_y_conserva_tarifa PASSED             [ 40%]
tests/integration/test_servicios.py::test_us15_kpis PASSED                                                    [ 41%]
tests/integration/test_servicios.py::test_sembrar_es_idempotente PASSED                                       [ 43%]
tests/unit/test_cargos_vehiculo.py::test_us10_retraso_exacto_no_cobra_y_un_minuto_cobra_una_hora PASSED       [ 45%]
tests/unit/test_cargos_vehiculo.py::test_us10_kilometraje_limite_1000_incluidos PASSED                        [ 46%]
tests/unit/test_cargos_vehiculo.py::test_us11_combustible_octavo_es_168_75 PASSED                             [ 48%]
tests/unit/test_cargos_vehiculo.py::test_us09_comparar_y_danos_nuevos PASSED                                  [ 50%]
tests/unit/test_cargos_vehiculo.py::test_estado_fisico_y_dinero_invalidos PASSED                              [ 51%]
tests/unit/test_cargos_vehiculo.py::test_us12_us13_ciclo_del_vehiculo PASSED                                  [ 53%]
tests/unit/test_cargos_vehiculo.py::test_vehiculo_transiciones_invalidas PASSED                               [ 55%]
tests/unit/test_inventario_reservacion.py::test_us02_us03_confirmar_genera_evento_y_bloquea_lugar PASSED      [ 56%]
tests/unit/test_inventario_reservacion.py::test_br02_sobreventa_rechazada PASSED                              [ 58%]
tests/unit/test_inventario_reservacion.py::test_br02_limite_reserva_que_empieza_el_dia_de_devolucion PASSED   [ 60%]
tests/unit/test_inventario_reservacion.py::test_br02_ocupacion_se_mide_dia_por_dia PASSED                     [ 61%]
tests/unit/test_inventario_reservacion.py::test_reservacion_no_se_confirma_dos_veces PASSED                   [ 63%]
tests/unit/test_inventario_reservacion.py::test_reservacion_rechaza_inventario_de_otra_categoria PASSED       [ 65%]
tests/unit/test_inventario_reservacion.py::test_inventario_validaciones_y_liberar PASSED                      [ 66%]
tests/unit/test_inventario_reservacion.py::test_periodo_invalido_y_categoria_desconocida PASSED               [ 68%]
tests/unit/test_renta.py::test_us04_us07_entrega_completa PASSED                                              [ 70%]
tests/unit/test_renta.py::test_rn02_entrega_bloqueada_informa_faltantes PASSED                                [ 71%]
tests/unit/test_renta.py::test_entrega_con_vehiculo_de_otra_categoria PASSED                                  [ 73%]
tests/unit/test_renta.py::test_no_se_entrega_dos_veces PASSED                                                 [ 75%]
tests/unit/test_renta.py::test_us14_upgrade_conserva_tarifa_y_entrega_categoria_superior PASSED               [ 76%]
tests/unit/test_renta.py::test_rn01_upgrade_rechazado_si_hay_unidades PASSED                                  [ 78%]
tests/unit/test_renta.py::test_rn01_upgrade_a_categoria_inferior_o_igual PASSED                               [ 80%]
tests/unit/test_renta.py::test_rn01_upgrade_solo_antes_de_entregar PASSED                                     [ 81%]
tests/unit/test_renta.py::test_us08_extension_con_disponibilidad PASSED                                       [ 83%]
tests/unit/test_renta.py::test_rn03_extension_rechazada_sin_capacidad PASSED                                  [ 85%]
tests/unit/test_renta.py::test_rn03_limite_otra_reserva_inicia_en_la_nueva_fecha_fin PASSED                   [ 86%]
tests/unit/test_renta.py::test_rn03_extension_que_no_amplia_o_antes_de_entregar PASSED                        [ 88%]
tests/unit/test_renta.py::test_us10_us11_devolucion_con_cargos_y_limpieza PASSED                              [ 90%]
```

Docker Test - 3

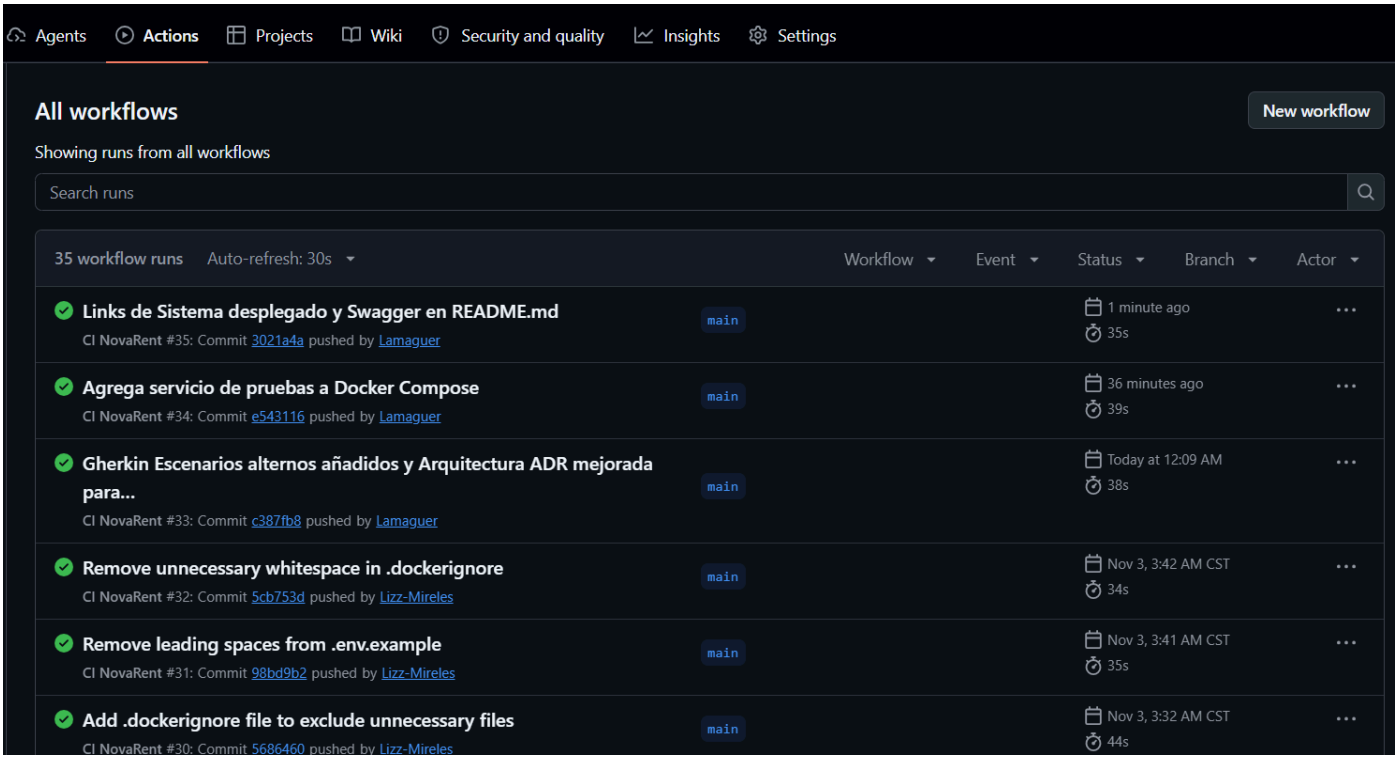
```
tests/unit/test_renta.py::test_devolucion_validaciones PASSED                                    [ 93%]
tests/unit/test_specifications.py::test_composicion_and_or_not PASSED                            [ 95%]
tests/unit/test_specifications.py::test_rn02_todos_los_requisitos PASSED                         [ 96%]
tests/unit/test_specifications.py::test_rn02_us07_lista_todo_lo_que_falta PASSED                 [ 98%]
tests/unit/test_specifications.py::test_rn01_upgrade_solo_sin_unidades_y_categoria_superior PASSED [100%]

============================================== warnings summary ===============================================
../usr/local/lib/python3.11/site-packages/fastapi/testclient.py:1
  /usr/local/lib/python3.11/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `s
tarlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

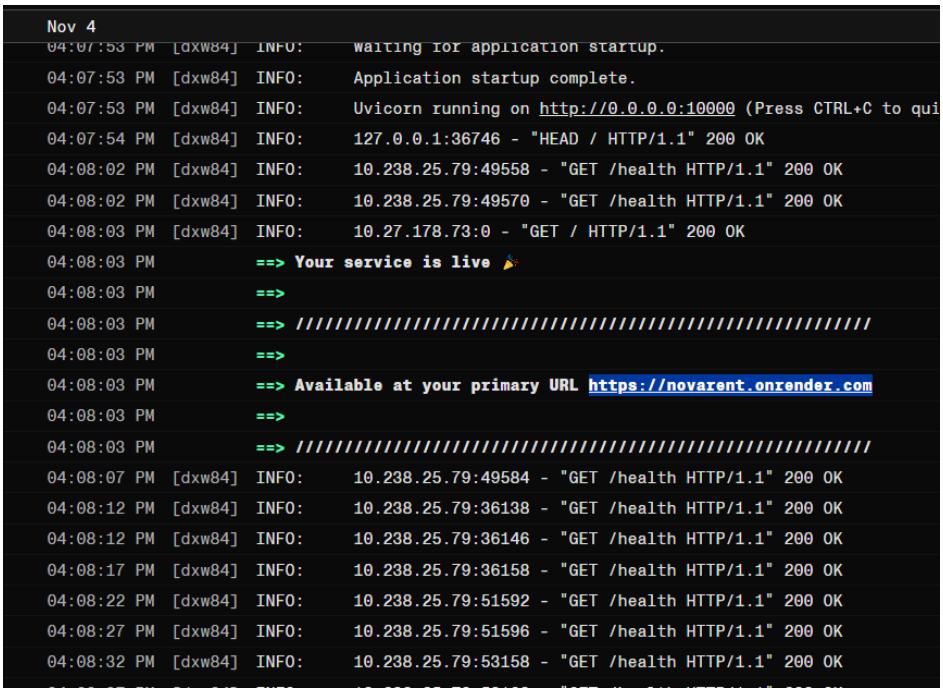
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================ tests coverage ================================================
_____________________________________ coverage: platform linux, python 3.11.18-final-0 _____________________________________

Name                                  Stmts   Miss  Cover
---------------------------------------------------------
src/__init__.py                           0      0   100%
src/api/__init__.py                       0      0   100%
src/api/main.py                         123      0   100%
src/api/mensajes.py                      24      2    92%
src/application/__init__.py               0      0   100%
src/application/servicios.py            188      2    99%
src/domain/__init__.py                    0      0   100%
src/domain/cargos.py                     43      0   100%
src/domain/errores.py                    10      0   100%
src/domain/eventos.py                     8      0   100%
src/domain/inventario.py                 29      1    97%
src/domain/politicas.py                  14      0   100%
src/domain/renta.py                      69      0   100%
src/domain/reservacion.py                22      0   100%
src/domain/specifications.py             64      1    98%
src/domain/valores.py                    55      0   100%
src/domain/vehiculo.py                   22      0   100%
src/infrastructure/__init__.py            0      0   100%
src/infrastructure/db.py                 20      1    95%
src/infrastructure/models.py             67      0   100%
src/infrastructure/pasarela.py            3      0   100%
src/infrastructure/repositorios.py      170      8    95%
src/infrastructure/seed.py               33      0   100%
src/infrastructure/unit_of_work.py       19      0   100%
---------------------------------------------------------
TOTAL                                   983     15    98%
Required test coverage of 80% reached. Total coverage: 98.47%
========================================== 60 passed, 1 warning in 29.80s ==========================================
PS C:\Users\Acer\Desktop\NovaRentU6>
```

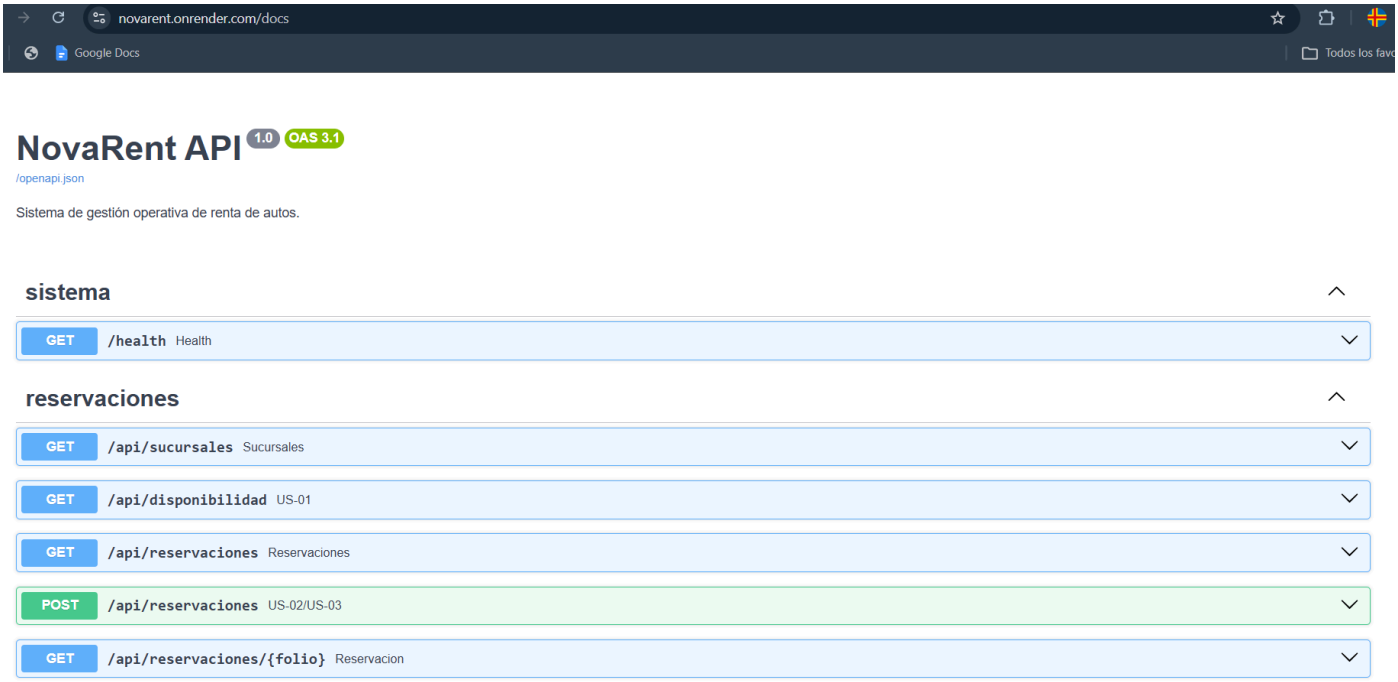
Actions en verde después de pushes



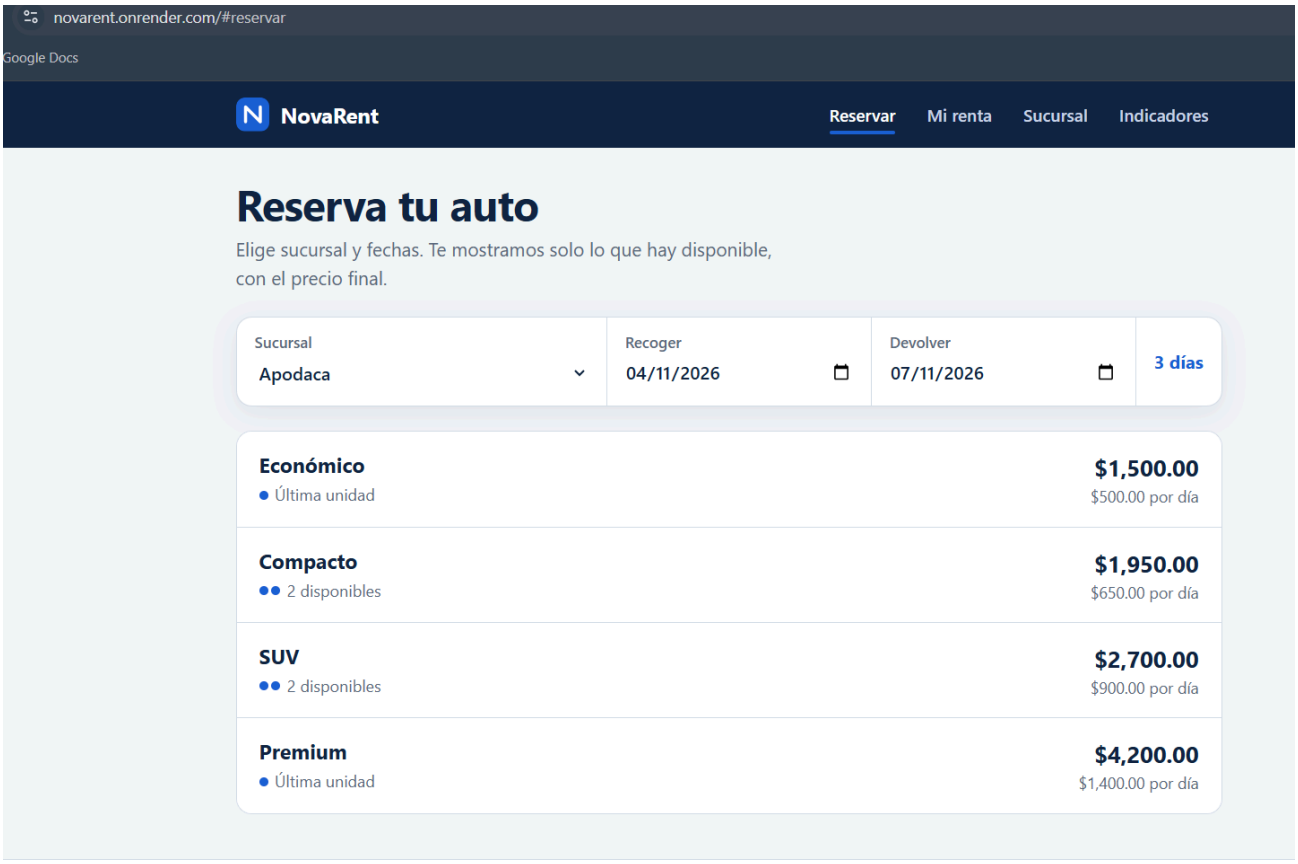
Render afirmando que la página está levantada/live



Nova Rent SWAGGER - LIVE



NovaRent principal - LIVE



Renta Reservada

Mi renta

Consulta el estado de tu renta, extiéndela o revisa tus cargos con tu folio.

Folio de reserva

F-68EF04CE

Buscar

F-68EF04CE

Reserva confirmada

A nombre de Castillo almaguer

Reservada — En renta — Devuelta

Tu reserva está confirmada. Presenta tu folio en la sucursal para recoger tu auto.

Categoría	Recoger	Devolver	Tarifa por día
Premium	**6 nov 2026**	**8 nov 2026**	**$1,400.00**

Renta total
$2,800.00

Registración de Venta

Entrega del auto

Sin identificación, licencia, pago y checklist inicial, la entrega se bloquea y te decimos qué falta.

- [x] Identificación oficial verificada
- [x] Licencia de conducir vigente

Tarjeta (se simula el depósito; no se guarda)

Tarjeta para el depósito

Checklist inicial

Combustible (octavos de tanque) 8/8

Kilometraje

12500

Daños visibles (opcional)

Por ejemplo, rayón en puerta | Agregar

Rayón en puerta × | Le faltan dos puertas. ×

Registrar entrega

Estado después de renta

F-68EF04CE — En renta

A nombre de Castillo almaguer

Reservada — En renta — Devuelta

Disfruta tu renta. Devuélvela el 8 nov 2026; después de esa fecha se cobra retraso por hora.

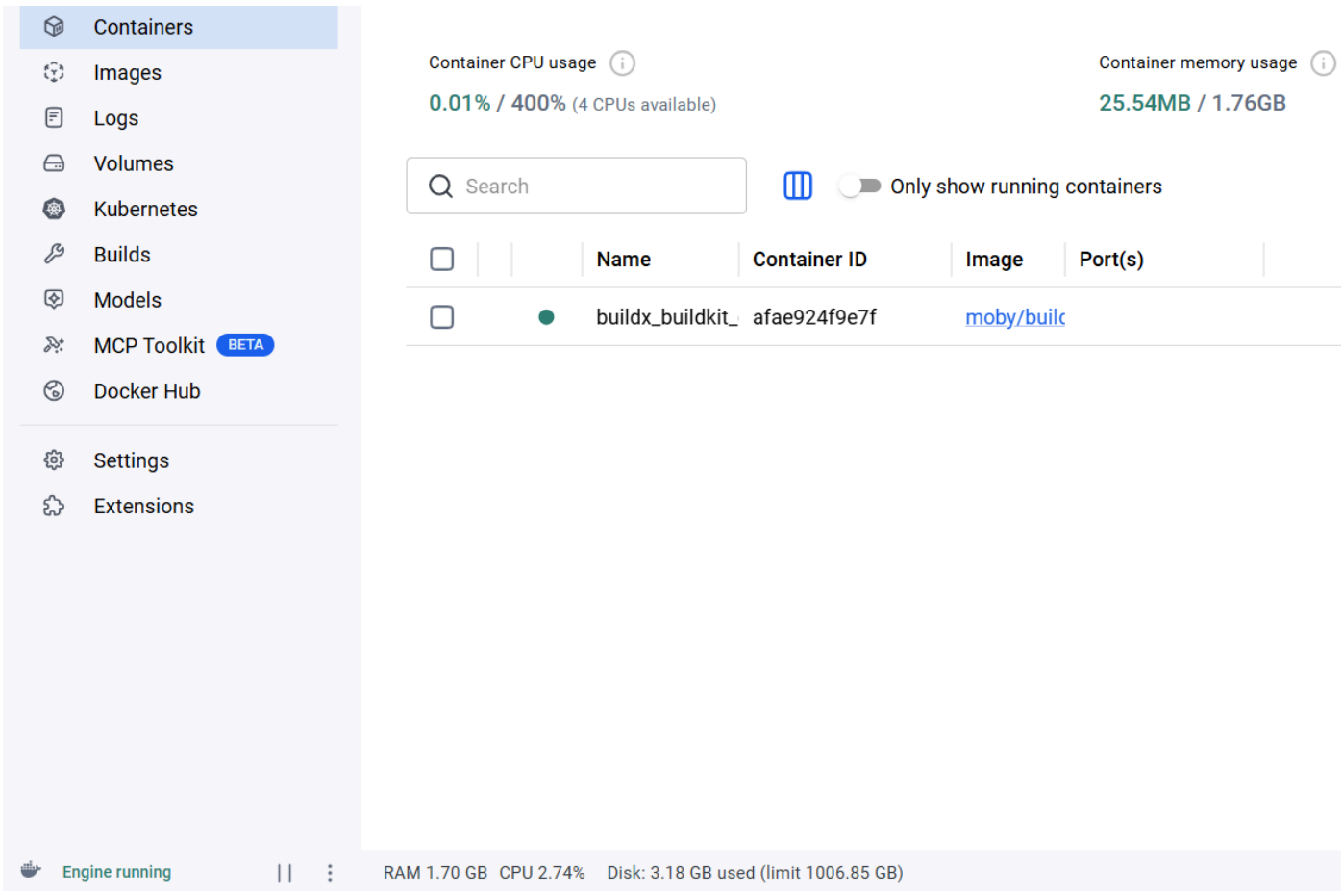
Categoría	Recoger	Devolver	Tarifa por día
Premium	**6 nov 2026**	**8 nov 2026**	**$1,400.00**

Vehículo	Renta total
MTY-CENTRO-PRE-01	**$2,800.00**

Renta Extendida



Docker corriendo



Limitaciones conocidas

Limitaciones conocidas y trabajo futuro	
Pasarela de pago simulada	El BRD no define proveedor; el sistema no mueve dinero. Una tarjeta que termina en 0000 se rechaza para fines de demostración.
Tarifario ficticio	El BRD supone una lista de precios oficial pero no la incluye (ADR-004). Los valores se reemplazan en un solo archivo.
Checklist sin fotos ni firma	US-06 y BR-01 piden fotos y firma del cliente. El sistema registra combustible, kilometraje y daños como texto, pero no captura fotografías ni firma.
Aviso a limpieza por panel, no por notificación	US-12 pide una alerta en tiempo real. El personal de limpieza ve la lista de vehículos pendientes al abrir su pantalla, pero no recibe una notificación automática.
Sin autenticación ni roles	Cliente, agente, limpieza y administración usan la misma interfaz sin inicio de sesión.
Fuera de alcance	Cancelación de reservaciones, seguros y conductores adicionales.
Concurrencia no probada con carga	La reserva usa bloqueo de fila en PostgreSQL (ADR-006), pero no se hicieron pruebas con peticiones simultáneas.
Sin migraciones de base de datos	Un cambio de columna exige recrear la base.
Infraestructura gratuita	El servicio de Render se duerme tras unos minutos sin tráfico y la base gratuita caduca a los 30 días de crearse (creada el 4 de noviembre de 2026).
Mejoras futuras	Fotos y firma, notificaciones, roles de usuario, migraciones, interfaz formal para la pasarela y pruebas de concurrencia.
Pruebas CSV	El paquete de datos de ejemplo (casos_prueba.csv, transiciones_prueba.csv) no se utilizó, porque el modelo de pruebas se derivó del BRD propio

Bitácora de IA

Se usó Claude como asistente para revisar la arquitectura, generar diagramas y apoyar la documentación; las decisiones se validaron con pruebas.

Casos	Corrección
(1)	Al integrar la persistencia, la revisión detectó que los IDs de vehículo usaban las 3 primeras letras de la sucursal y dos sucursales generaban el mismo ID; se corrigió.
(2)	Un diagrama generado nombraba clases que no existían en el código (Compuesta_AND_OR); se corrigió para que coincida con el código.

Conclusiones

Ha sido un largo recorrido a través de las unidades para llegar hasta el lugar que el proyecto que se nos fue dado en la Unidad 2 esta, despues de meses aprendiendo la lógica de negocios y como está es fundamental a la hora de crear uno de estos se aprendió con el tiempo como es la manera correcta de compartir trabajo (git/github) como funcionan los apis (FastAPI) y como no solo hacer tests sino tests automáticos después de cada push al proyecto, gracias a la Unidad 3, 4 y 5 tenemos un proyecto extremadamente fundamentado sólido y funcional con un link real gracias a Docker Desktop y especialmente a la sesión gratuita de Render, este proyecto NovaRent es solo un ejemplo ficcional al no ser un negocio de verdad pero el hecho y el proceso que nos tomó en hacerlo nos muestra cómo sería el “deployment” de un negocio de verdad, de una página de este, puede que el proceso no haya sido perfecto, claro, tenemos mucho que aprender todavía pero este solo es el inicio, ya que lo vimos fueron los fundamentos de la lógica de negocio.

El defecto más costoso sería una sobreventa: el cliente llega y no hay auto. El sistema lo evita contando lugares por día y bloqueando la fila del inventario al reservar, y lo verifica la prueba test_br02_sobreventa_rechazada_y_nada_se_guarda. También aprendimos que las reglas del BRD deben convertirse en pruebas automáticas, que cada cambio se valida con el pipeline, y que un sistema solo está terminado cuando corre en una URL pública.